

Coding style & conventions

1. **Zero-warning policy:** formatting, casing, spacing, and bracket placement are strictly enforced by StyleCop. If the code builds with zero warnings, the syntax is compliant
2. **Layer separation:** services and repositories **must never reference UI namespaces** (eg `Microsoft.UI.Xaml`). **Do not** return UI types like `Visibility/Color` from a service. Services return raw data, `ViewModels` decide how to display it
3. **No heavy lifting in constructors:** constructors must only be used for assigning injected dependencies. Database queries or heavy API calls must be placed in a separate `Load()` or `Initialize()` method
4. **Fail fast:** **do not** nest the entire method inside a massive `if (isValid {...})` block. Put the error checks at the very top of the method and return or throw immediately(`!isValid {throw error}`)
5. **Boolean naming:** any variable, property, or method returning a boolean must be phrased as a question using prefixes like `Is`, `Has`, `Can`, or `Should` (e.g., `HasFlights`, **not** `CheckFlights`)
6. **No empty catch blocks:** empty `catch { }` blocks are **forbidden**. If you catch an exception to prevent a crash, you must either log it, add a comment explaining exactly why it is safe to ignore, or wrap it and throw it
7. **Method names:** every method name must start with a **verb** representing an action (`CalculateTotal`, `GenerateFlightCode`)
8. **Class names:** every class name must be a **noun** representing an entity (`AirportRepository`). **Avoid "Manager" or "Processor"** as they are non-semantic and vague
9. **Enum domain mapping:** Distinct entity states (like `EmployeeRole`) must be mapped to strongly-typed C# Enums. Never rely on raw string comparisons for business logic
10. **Single responsibility for methods:** a method should do **one thing**. If you find yourself using the word "And" in a method name (e.g., `SaveAndRefresh`), the method should likely be split into two
11. **Comments:** code should be self-documenting regarding *what* it is doing (via semantic naming). Comments should only be used to

explain business intent or complex workarounds (eg // Subtracting 1 hour to handle local timezone from the DB)

12. **The 3-strike indentation rule:** methods must not exceed **3 levels of indentation**. If you find yourself nesting an if inside a foreach inside another if, the inner logic must be extracted into a private, semantically-named method. This keeps methods short and readable
13. **Use string interpolation over concatenation:** avoid using **+** to build strings or **string.Format()**. Use **string interpolation** (**\$"Code: {code}"**) for better readability and to prevent the "magic string" mess that often occurs with complex path or query building
14. **No abbreviations:** variables, methods, and classes must use full, semantic words (e.g., **employeeRepository** instead of **empRepo**). The only allowed exception is **Id** as an industry standard
15. **No magic numbers/strings:** literal strings and numbers used in business logic must be declared as explicitly named **const** variables at the top of the file or class
16. **No complex lambdas:** avoid chaining functional LINQ methods (like **.Where().Select()**). Use explicit **foreach** loops to make logic readable and easy to debug
17. **Primary constructors:** use .NET 10 primary constructor syntax for all dependency injection to eliminate constructor boilerplate and variable shadowing
18. **Enum domain mapping:** distinct entity states (like Employee Roles) must be mapped to strongly typed C# **Enums** rather than relying on raw string database comparisons
19. **Test naming convention:** all unit tests must be named using the 3-part pattern: **[MethodName]_[ExpectedBehavior]_[Condition]**
20. **The source pattern for mocks:** when setting up moq objects, the mock variable must describe its state. Same when writing the lambda inside the setup(**don't: source => source.GetAll()**
do:getFlightsWithConflicts())

